GitHub 新手学生使用指南

—— 基于 Git 的代码托管与协作入门

日期：2026 年 10 月

目录

一、Git 与 GitHub 是什么 4
（一）Git：本地版本控制工具 4
（二）GitHub：代码托管与协作平台 4
（三）Git 与 GitHub 的关系 4
二、注册与基础配置 4
（一）注册 GitHub 账号 4
（二）安装 Git 并配置身份 5
（三）配置 SSH 密钥（推荐） 5
（四）小贴士：网络访问不畅怎么办 6
（五）学生认证：免费领取教育福利 6
三、核心概念速查 7
阅读路线：三种使用姿势，对号入座 8
四、电脑端与手机端：两个客户端怎么用 8
（一）电脑端：GitHub Desktop 8
（二）手机端：GitHub App 11
（三）三端怎么配合 12
（四）学习进度自检 12
（五）新手最容易漏掉的四件事 13
五、新手最常用的操作流程 14
（一）Git 的四个工作区域 14
（二）10 分钟第一次实操 15
（三）把自己的项目传到 GitHub 16
（四）下载别人的项目 17
（五）日常修改代码并提交 17

（六）参与开源项目：标准 PR 流程.....17
（七）撤销与回退：改错了怎么办.....18
六、提升效率的实用功能.....19
（一）README.md：仓库的门面.....19
（二）.gitignore：排除不需要跟踪的文件.....19
（三）GitHub Actions：自动化流水线.....19
（四）GitHub Pages：免费托管静态网页.....19
七、常见问题与避坑.....20
（一）不要把密钥和密码提交到仓库.....20
（二）push 被拒绝（rejected）怎么办.....20
（三）提交信息要写清楚.....20
（四）clone 慢或失败怎么办.....21
（五）Windows 换行符提示：CRLF 警告.....21
（六）桌面端与手机端专属问题.....21
附录：新手常用命令速查表.....23

一、Git 与 GitHub 是什么

（一）Git：本地版本控制工具

Git 是一种分布式版本控制工具，安装在电脑上本地运行。它会记录项目每一次代码修改的“快照”，让你随时查看改动、回到任意历史版本，并支持在分支上并行开发。Git 本身不需要联网，是后续所有操作的基础工具。

（二）GitHub：代码托管与协作平台

GitHub 是建立在 Git 之上的代码托管与协作网站。你可以把本地仓库同步到 GitHub 的云端仓库，实现多设备同步、多人协作，并借助 Issue（问题讨论）、Pull Request（合并请求）、GitHub Actions（自动化）等功能完成开源项目的协作。

（三）Git 与 GitHub 的关系

简单说：Git 负责“管版本”，GitHub 负责“托管与协作”。Git 是装在电脑上的工具，GitHub 是存放远程仓库的平台。日常流程是：先在本地用 Git 提交修改，再推送到 GitHub；从 GitHub 拉取更新，再继续本地开发。两者的关系如表 1 所示。

表 1　Git 与 GitHub 对比

比较项	Git	GitHub
本质	本地运行的版本控制工具	基于 Git 的代码托管与协作网站
作用	记录每次修改，支持回退与分支	存放远程仓库，多人协作与代码评审
是否依赖网络	本地离线即可使用	需要联网访问
主要使用方式	命令行（git 命令）	网页操作配合 git 命令

二、注册与基础配置

（一）注册 GitHub 账号

访问 github.com 注册账号。用户名只能使用字母、数字和连字符（-），无法使用中文；它会出现在你所有仓库的地址中（例如 github.com/用户名/仓库名），注册后不方便随意更改，注册前先想好。

（二）安装 Git 并配置身份

在本机安装 Git（官方下载地址 git-scm.com）。如果已经装了 GitHub Desktop，也不能跳过这一步：桌面端自带的那份 Git 只给它自己用，没加进系统路径——在终端里敲 git 会提示「不是内部或外部命令」。要用命令行，必须单独安装 Git（安装向导里的 PATH 选项保持默认，装完重开一个终端）。安装完成后打开终端，先告诉 Git 你的名字和邮箱，这些信息会记录在每一次提交中：

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
```

（三）配置 SSH 密钥（推荐）

SSH 密钥可以让你免密推送代码。在终端执行以下命令生成密钥（-C 后填注册邮箱）：

```
ssh-keygen -t ed25519 -C "你的邮箱"
```

一路按回车使用默认保存位置即可。然后查看并复制公钥内容（Windows 用户请在 Git Bash 中执行以下命令；若使用 PowerShell，可改用 type $env:USERPROFILE\.ssh\id_ed25519.pub）：

```
cat ~/.ssh/id_ed25519.pub
```

复制输出的整段内容，粘贴到 GitHub 的 Settings（设置）→ SSH and GPG keys → New SSH key，填写一个便于识别的标题（如“我的电脑”）后保存。配置完成后即可使用 SSH 地址免密推送。

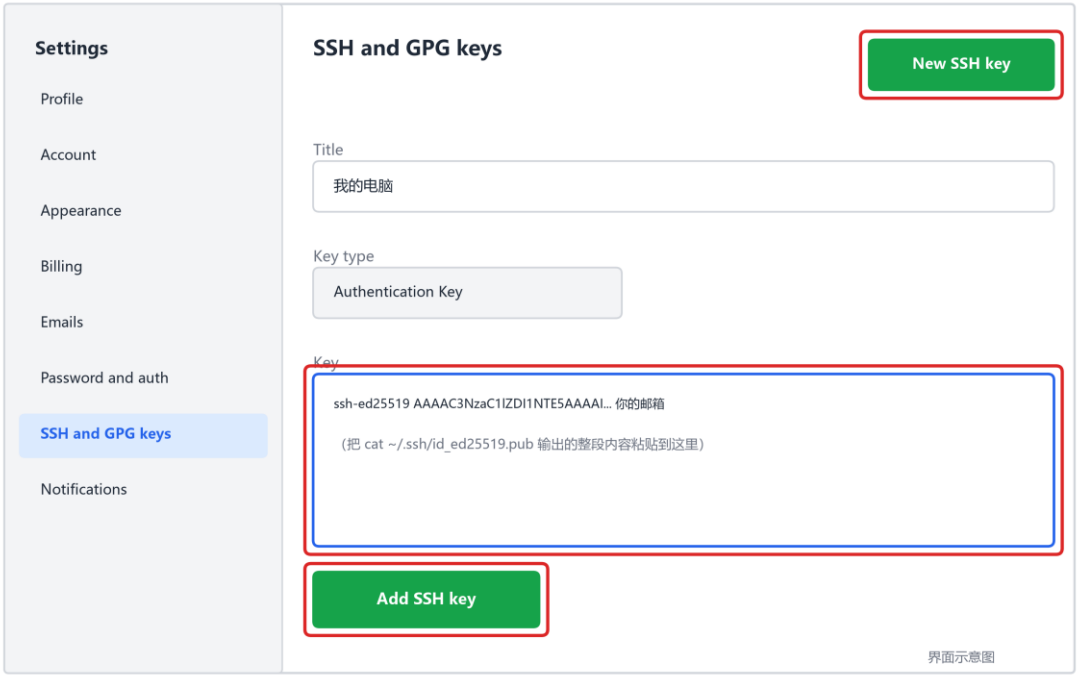


图 1　添加 SSH 公钥的页面位置

（四）小贴士：网络访问不畅怎么办

访问 github.com 偶有不稳定是常见现象。若使用 SSH（默认 22 端口）推送失败，可按 GitHub 官方说明改用 SSH over 443：在 ~/.ssh/config 文件中写入以下内容：

```
Host github.com
  Hostname ssh.github.com
  Port 443
  User git
```

保存后用 ssh -T git@github.com 测试连通性。此外，克隆和推送也可以使用 HTTPS 地址（https://github.com/用户名/仓库名.git），推送时按提示输入用户名和访问令牌（Personal Access Token，在 Settings → Developer settings 中创建），密码已不再被支持。

（五）学生认证：免费领取教育福利

作为在校学生，完成 GitHub 学生认证后即可免费领取 **GitHub Student Developer Pack（学生开发者包）**，其中通常包含 GitHub Copilot Pro 编程助手、

云服务额度以及多家合作方提供的开发工具与教程资源，具体内容以申请页面当期显示为准。

认证步骤如下：

- 登录 GitHub，访问 education.github.com，选择学生身份开始申请；
- 填写学校名称、预计毕业时间等基本信息；
- 提供在读证明：有学校邮箱（.edu.cn 等）可直接用邮箱验证；没有学校邮箱的同学，可按页面提示上传学生证照片，或上传学信网（CHSI）在线验证报告；
- 提交后等待审核，通常几个工作日内出结果；通过后在教育页面即可领取各项福利。

小提醒：申请信息务必真实，部分福利在毕业后会陆续到期，建议在校期间尽早领取。

三、核心概念速查

开始操作前，先记住 GitHub 中最常用的几个概念，见表 2。

表 2　GitHub 核心概念速查

概念	含义	一句话理解
Repository（仓库）	一个项目的代码及全部历史记录	GitHub 的基本单位，一个项目对应一个仓库
Branch（分支）	从主线上分出的独立开发线	main 是默认主分支，新功能在分支上开发
Commit（提交）	一次代码修改的快照	每次保存都带一句说明，可随时回退
Pull Request（PR）	请求把自己的分支合并进主分支	协作的核心，合并前可被他人审查
Fork（复刻）	把别人的仓库复制到自己账号下	在副本上修改，不直接影响原仓库
Issue（问题）	仓库里的问题与需求讨论区	用来报 bug、提需求、讨论方案
Star（星标）	对仓库的收藏与点赞	表达关注与支持，方便日后查找

阅读路线：三种使用姿势，对号入座

这份指南同时覆盖网页、桌面客户端、手机客户端和命令行，但没有人需要一次全学。按你的使用习惯选一条线即可。

A 线丨只用桌面端（多数同学适用）：读完第一、二、三章打底，直接进入第四章客户端，跟着做完第一次实操；第五章命令行先放着，需要批量处理文件、在服务器上操作或想看懂别人给的命令时再回来。遇到问题先查第七章常见问题，其中（六）节专门收集桌面端与手机端的问题。

B 线丨想认真学命令行：读完前三章后，第四章客户端只需浏览第（一）节了解 Desktop，然后进第五章从头跟做；附录命令速查表贴在手边，用上三次就熟了。

C 线丨手机应急为主：读完第一、三章的概念，看第四章（二）手机端小节即可。记住一个原则：手机上改了文件，电脑端必须 Pull 一次才会同步。

无论哪条线，第二章的 SSH 配置和学生认证都建议抽时间完成。A 线读者可以把表 3（Desktop 按钮状态）截图留在手机里，比背命令表实用。

四、电脑端与手机端：两个客户端怎么用

前面几节的操作主要发生在网页和命令行里。GitHub 另外提供两个官方客户端——电脑端的 GitHub Desktop 和手机端的 GitHub App。二者定位不同，不能互相替代：桌面端解决“把代码搬到本地改”，手机端解决“随时随地看和回”。

（一）电脑端：GitHub Desktop

1. 下载与安装

访问 desktop.github.com，页面会自动识别操作系统：Windows 用户下载 .exe 安装包（要求 Windows 10 及以上），macOS 用户下载 .dmg。安装过程一路默认即可，完成后启动客户端。

2. 首次登录

点击 Sign in to GitHub.com，浏览器会自动弹出 GitHub 授权页面，确认后点 Authorize desktop。页面显示“OAuth application authorized”即表示授权完成，客户端随之进入主界面。整个过程不需要在客户端里重新输入密码，凭据由系统统一保存，之后推送不必反复验证。

3. 首次进入的四个入口

- Create a tutorial repository：创建一个官方教学仓库，用来练习分支与提交，会在你的账号下真实建一个仓库。
- Clone a repository from the Internet：把 GitHub 上已有的仓库复制到本地。
- Create a New Repository on your local drive：在本地新建仓库，建好后可以发布到 GitHub。
- Add an Existing Repository from your local drive：把本地已经存在的 Git 仓库挂进客户端管理。

若界面提示“Looks like there are no repositories...”，说明账号下暂时没有远端仓库，这不是出错，建好并发布第一个仓库后列表就会显示。

4. 界面与按钮

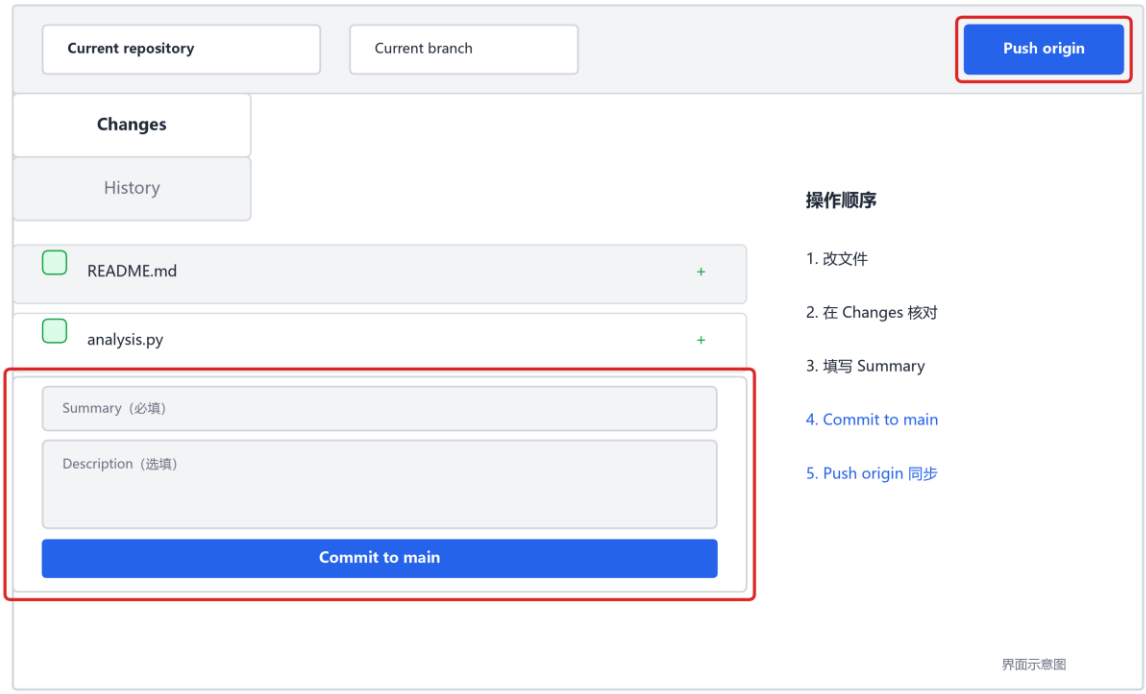


图 2　GitHub Desktop 主界面

左上角 Current repository 切换仓库，右侧 Current branch 切换分支；左侧 Changes 列出尚未提交的改动，History 查看历史提交；左下角 Summary 必填、Description 选填，写完点 Commit to main 完成一次提交。右上角按钮会随仓库状态变化，含义见表 3。

表 3　GitHub Desktop 右上角按钮的四种状态

按钮	含义	该做什么
Publish repository	仓库只存在于本地，尚未上传	点击发布到 GitHub
Fetch origin	本地与云端一致	无需操作
Push origin	有本地提交待推送	点击推送
Pull origin	云端有新提交	点击拉取

5. 日常流程

改文件 → 在 Changes 中核对改动 → 填写 Summary → Commit to main → Push origin。本地提交不会因为断网或关机丢失，只有执行 Push 之后才真正同步到云端，这一点与命令行完全一致。

6. 局限

遇到合并冲突、需要 rebase、要撤销已经推送的提交时，Desktop 只能处理其中一部分，仍需回到命令行。客户端只是 Git 的图形外壳，不是替代品。

（二）手机端：GitHub App

1. 下载

iOS 用户在 App Store 搜索 GitHub，Android 用户在 Google Play 搜索 GitHub。需要注意：中国大陆区的 App Store 不提供该应用，需要切换到其他地区的 Apple ID 才能下载；Android 端若没有安装 Google Play，可按 GitHub 官网移动端页面给出的指引获取安装包。

2. 登录

输入账号与密码后，还需通过双因素验证（2FA），填入验证器生成的 6 位验证码；支持 passkey 的设备可以直接用指纹或面容登录。GitHub App 本身也可充当 2FA 验证器，开机换机时比较方便。

3. 能做什么

浏览与搜索仓库、查看代码、Star 与 Watch、查看并回复 Issue 和 Pull Request、合并 PR、直接修改单个文件并提交、查看 Actions 运行结果、接收通知。

4. 不能做什么

不能把仓库克隆到本地、不能运行或调试代码、不能解决合并冲突、不能批量处理文件。手机端是“查看与轻量修改”的工具，不是开发环境。

5. 一个常见误解

在手机上改完文件并提交后，电脑上的本地副本不会自动更新，必须在桌面端或命令行执行一次 Pull，才能拿到最新版本；反过来在电脑上推送之后，

手机端也要下拉刷新才会看到变化。三端之间靠“提交”和“拉取”对齐，不靠实时同步。

（三）三端怎么配合

网页端、桌面端、手机端各有擅长的场景：网页端适合管理仓库设置、发起和审查 PR；桌面端适合真实写代码、批量提交；手机端适合通勤路上处理通知和应急修改。三者的能力对比如表 4。

表 4　网页端、桌面端、手机端能力对比

能力	网页端	桌面端	手机端
浏览代码与提交历史	支持	支持	支持
新建仓库	支持	支持（本地新建后发布）	支持（网页式新建）
修改并提交单个文件	支持	支持	支持
整目录提交、批量改动	不支持	支持	不支持
克隆到本地	不支持	支持	不支持
解决合并冲突	有限	有限	不支持
查看与重跑 Actions	支持	仅查看	仅查看
处理 Issue 与 PR	完整	完整	基本

（四）学习进度自检

把前面几节的操作拆成清单，逐项对照，就能知道自己停在哪一步，见表 5。

表 5 是作者本人按本指南实际操作的进度记录，供读者参照；你也可以复制该表，把状态改为自己的实际情况，逐项打勾。

表 5　学习进度自检（截至 2026 年 10 月）

阶段	内容	状态
账号	注册 GitHub 账号	已完成
基础配置	安装 Git、配置提交身份	已安装（未进 PATH，命令行暂不可用）
基础配置	配置 SSH 密钥	待完成（当前使用 HTTPS 地址）
账号安全	开启双因素验证	待确认
教育福利	学生认证与学生开发者包	待申请
网页端	新建仓库、修改文件、提交	未进行（仓库由桌面端创建）
桌面端	下载、登录、新建仓库、发布、提交、推送	已完成（2026 年 10 月 28 日）
桌面端	克隆他人仓库、处理冲突	待练习
手机端	下载并登录	已完成
手机端	修改文件、查看 Issue 与 PR	待练习
协作	Fork 与 Pull Request 完整流程	待练习

（五）新手最容易漏掉的四件事

第一，双因素验证要尽早开启。只靠密码一层防护，一旦密码泄露，仓库内容全部暴露。建议登录后进入 Settings → Password and authentication，绑定验证器或手机短信。

第二，SSH 密钥没有配置。现在推送依赖客户端保存的 HTTPS 凭据，换一台电脑就要重新授权；配置好 SSH 之后，命令行推送可以免密，在服务器上操作也方便。配置方法见第二节第（三）部分。

第三，学生认证尚未申请。GitHub Student Developer Pack 包含 Copilot Pro、云服务额度等福利，在校期间申请最划算，毕业后部分权益会到期。

第四，提交身份建议统一。执行 git config --global user.name 查看当前提交作者名。建议填网名或拼音缩写，不要写真实姓名——这个名字会永久写进每一次提交，公开仓库里人人可见；提交之后再改，就要重写历史。

五、新手最常用的操作流程

本章讲的是命令行操作，属于第四章客户端之外的进阶内容，可按需延后阅读。如果你只用 GitHub Desktop，下面这些步骤在客户端里都有对应按钮——新建仓库对应 Create a New Repository，提交对应 Commit to main，推送对应 Push origin——建议先在第四章用客户端跑通一次，再决定要不要学本章；等你需要批量处理文件、在服务器上操作，或者想看懂别人给的命令时，本章就是你要的内容。

（一）Git 的四个工作区域

在动手敲命令之前，先理解 Git 管理文件的四个工作区域，这能解释后面每条命令到底在做什么。工作区是你在电脑上实际编辑文件的文件夹；暂存区是一块临时存放区，git add 会把改动放进这里，让你先挑选本次要提交的内容；本地仓库保存所有已提交的版本快照，git commit 会把暂存区的内容固化为一次提交；远程仓库位于 GitHub 上，git push 才会把本地提交同步上去。四个区域的关系和常用命令如图 3 所示。



图 3　Git 四个工作区域与数据流向

（二）10 分钟第一次实操

下面用一个最小例子，把从建仓库到网页看到文件的完整流程串起来。建议第一次严格照做一遍，整体顺序如图 4。



图 4　10 分钟第一次实操（端到端闭环）

如果你使用 GitHub Desktop，上述每一步都有对应按钮：在网页或客户端新建仓库后，把文件放进文件夹，在 Changes 中核对改动、填写 Summary、点 Commit to main，再点 Publish repository 或 Push origin 即可，效果完全相同。

（三）把自己的项目传到 GitHub

在 GitHub 网页上新建一个空仓库（New repository），然后在本地项目文件夹内执行：

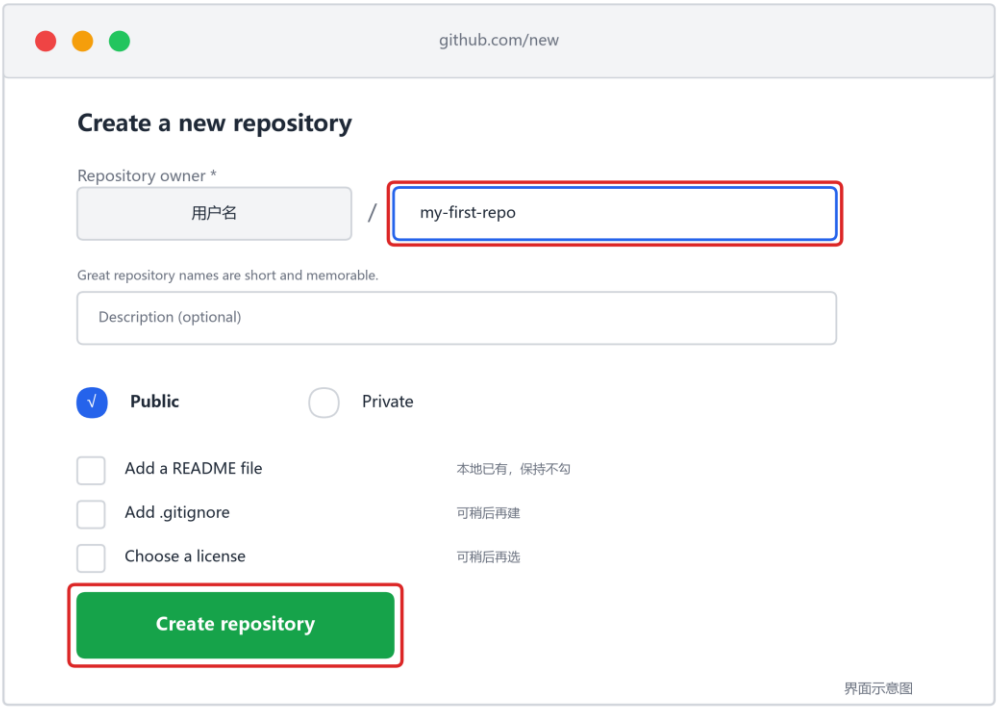


图 5　网页新建仓库界面

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:用户名/仓库名.git
git push -u origin main
```

说明：git init 把当前文件夹初始化为 Git 仓库；origin 是远程仓库的别名，可换成你实际的 SSH 地址；-u 参数让本地分支与远程 main 分支建立关联，之后直接执行 git push 即可。如果对命令行还不熟悉，也可以先用 GitHub

Desktop 图形客户端完成上述流程，它把常用的提交、推送做成了按钮，熟练之后再回到命令行也不迟。

（四）下载别人的项目

克隆（clone）会把整个仓库连同历史记录复制到本地：

```
git clone git@github.com:用户名/仓库名.git
```

也可以使用 HTTPS 地址：git clone https://github.com/用户名/仓库名.git，两种方式效果相同。

（五）日常修改代码并提交

每次动手修改前先拉取最新代码，修改完成后再提交推送：

```
git pull
git add 修改的文件
git commit -m "说明改了什么"
git push
```

（六）参与开源项目：标准 PR 流程

PR（Pull Request）是参与开源协作的标准方式，完整流程如下：

在 GitHub 网页上打开目标仓库，点击 Fork，把仓库复制到自己账号下。

克隆自己的 fork，并新建一个功能分支：

```
git clone git@github.com:你的用户名/仓库名.git
git checkout -b fix-xxx
```

修改代码，提交并推送到自己的 fork：

```
git add .
git commit -m "fix: 修复了 xxx 问题"
git push -u origin fix-xxx
```

回到 GitHub 网页，在目标仓库页面点击 Compare & pull request，填写修改说明后提交 PR，等待维护者审查与合并。

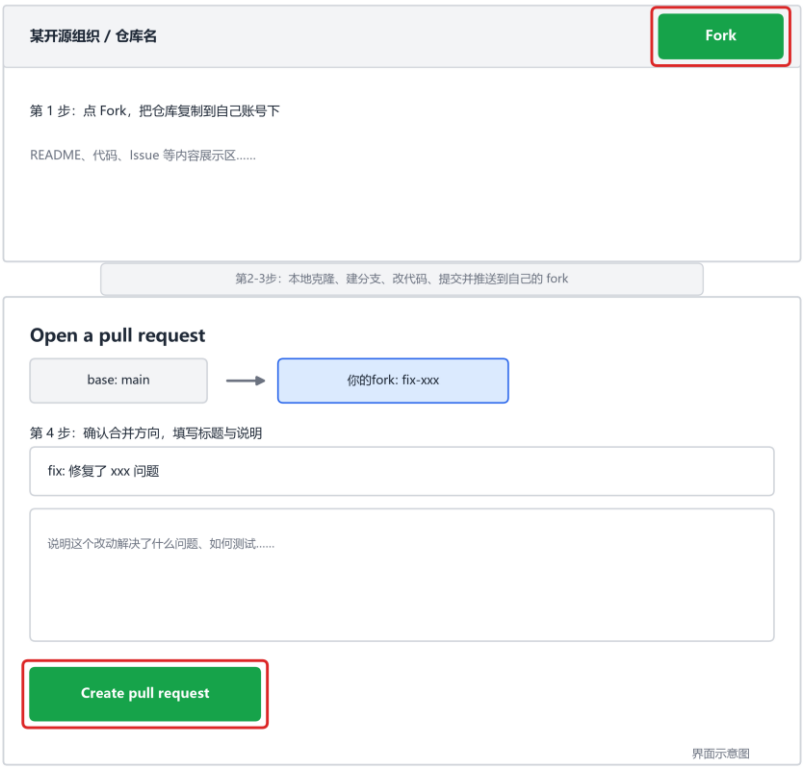


图 6　Fork 与 Pull Request 界面

分支命名常用 fix（修复）、feat（新功能）、docs（文档）等前缀，一目了然。

Fork 之后还有一个常见环节：原仓库会持续更新，你的副本需要定期把原仓库的最新改动同步过来，否则在提交 PR 时容易产生冲突。先把原仓库地址记为 upstream，再拉取并合并：

```
git remote add upstream git@github.com:原作者/仓库名.git
git fetch upstream
git checkout main
git merge upstream/main
```

合并后再执行 git push，自己的 fork 就和原仓库保持一致。upstream 只需添加一次，之后重复 fetch 与 merge 即可。

（七）撤销与回退：改错了怎么办

新手难免改错，下面按场景给出最常用的撤销方法，这些操作只影响本地，不会改动远程仓库。

第一，改乱了某个文件，想丢弃工作区的改动、恢复到上次提交的样子：

```
git restore 文件名
```

第二，git add 之后发现暂存错了文件，想把它移出暂存区，文件本身不会丢失：

```
git restore --staged 文件名
```

第三，最近一次提交的说明写错了，或漏提交了文件，修改后重新提交：

```
git commit --amend -m "修正后的说明"
```

第四，想回到历史上的某次提交。先用 git log --oneline 查看提交号，再按需回退：

```
git reset --hard 提交号
```

注意 reset --hard 会丢弃该提交之后的所有改动，执行前务必确认；仅想撤销暂存、保留文件内容时使用默认的 git reset --mixed。

六、提升效率的实用功能

（一）README.md：仓库的门面

README.md 是仓库首页直接展示的文件，用 Markdown 编写，介绍项目是什么、如何安装与使用，是别人了解你项目的第一入口，建议每个仓库都写好。

（二）.gitignore：排除不需要跟踪的文件

在仓库根目录创建 .gitignore 文件，把 node_modules/、.env、编译产物、临时文件等写入其中，它们就不会进入版本管理，也避免敏感信息被误提交。

（三）GitHub Actions：自动化流水线

在仓库的 .github/workflows/ 目录下编写 YAML 文件，可以在每次推送或合并时自动运行测试、构建和部署，实现“代码推上去，检查自动跑”。

（四）GitHub Pages：免费托管静态网页

在仓库 Settings → Pages 中开启，发布源选择“Deploy from a branch”（从分支部署）并选定分支，即可把静态页面免费托管上线：项目页地址形如用户名.github.io/仓库名；若把仓库直接命名为 用户名.github.io，则访问地址就

是 用户名.github.io，适合做个人主页。也可以选择 GitHub Actions 方式自定义构建流程。

七、常见问题与避坑

（一）不要把密钥和密码提交到仓库

API 密钥、密码、私钥一旦 push 就会永久进入历史记录。首先用 .gitignore 排除 .env 等文件，避免它们进入仓库；若已经泄露，第一步应立即在 GitHub 上删除或重置对应的密钥、令牌，再考虑清理历史：git filter-repo 需要单独安装（并非 Git 内置命令），且要重写历史并强制推送，会影响协作者；对个人仓库而言，最稳妥简单的办法是删除仓库重新创建。

（二）push 被拒绝（rejected）怎么办

push 被拒绝通常是因为远程分支有新提交，本地落后了。先执行 git pull --rebase 把远程更新合并进来，再重新推送；若提示冲突，手动解决冲突文件后执行 git add、git commit 完成合并，再 git push。

如果 git pull --rebase 或 merge 提示冲突，打开冲突文件，会看到类似下面这样的标记，Git 用它标出两个版本不一致的地方：

```
<<<<<<< HEAD
print("content from current branch")
=======
print("content from remote branch")
>>>>>>> 分支名
```

其中 <<<<<<< 到 ======= 之间是你当前的版本，======= 到 >>>>>>> 之间是对方的版本。手动保留正确内容或合并两者，删除这三行标记，保存后执行 git add 文件名、git commit（rebase 时执行 git rebase --continue），再重新推送即可。

（三）提交信息要写清楚

规范的提交信息能帮助自己和协作者理解改动。推荐约定式写法：fix: 修复登录页空指针、feat: 新增导出功能、docs: 更新使用说明，比笼统的 update 有用得多。

（四）clone 慢或失败怎么办

先检查网络是否稳定；可以改用 HTTPS 地址克隆。若 SSH 22 端口连接失败，按第二部分“小贴士”配置 SSH over 443 后重试。

（五）Windows 换行符提示：CRLF 警告

Windows 用户第一次提交时，可能看到 warning: LF will be replaced by CRLF 的提示。这是因为 Windows 与 Linux/macOS 的换行符不同，前者为 CRLF，后者为 LF，Git 会自动转换，属于正常提示，不影响使用，无需处理。若想统一为 LF，可在仓库根目录添加 .gitattributes 文件并写入 * text=auto eol=lf。

（六）桌面端与手机端专属问题

1. 桌面端遇到合并冲突怎么办

发生冲突时，Changes 列表中的文件会显示黄色警告图标。逐个打开冲突文件，编辑器里同样会看到 <<<<<<< 与 >>>>>>> 标记（含义见本章（二））；解决后回到客户端点 Continue merge 完成合并。想整文件保留某一方的版本，可右键该文件选择对应版本。如果冲突涉及大量文件，建议回到命令行处理，方法见第五章。

2. 手机上提交的文件，电脑上怎么看不到

手机端提交的是云端版本，电脑上的本地副本不会自动更新：桌面端点一次 Pull origin（或命令行 git pull）即可拿到；反过来电脑推送后，手机端下拉刷新即可看到。三端靠“提交”和“拉取”对齐，不靠实时同步。

3. 换电脑后怎么继续用

在新电脑安装 GitHub Desktop 并登录同一账号，用 Clone a repository from the Internet 把仓库克隆下来即可继续；若之前用命令行，需要在新电脑重新配置 SSH 密钥（方法见第二章第（三）部分）或改用 HTTPS 加访问令牌。旧电脑若不再使用，建议在 Settings → SSH and GPG keys 中删除旧公钥，并在 Settings → Developer settings 中吊销旧令牌。

把第一个项目成功 push 到 GitHub，你就迈出了开源协作的第一步。之后遇到问题，优先查看 GitHub 官方文档（docs.github.com），它覆盖了最权威的操作说明。

入门之后，推荐三类资源继续学习：一是 GitHub 官方交互式课程 skills.github.com，在真实仓库里用任务引导练习，适合巩固操作；二是《Pro Git》中文版（git-scm.com/book/zh），系统讲解版本控制原理，免费可读；三是 GitHub 官方文档 docs.github.com，遇到具体问题时优先查阅。

附录：新手常用命令速查表

目的	命令
初始化本地仓库	`git init`
查看当前状态	`git status`
暂存修改	`git add 文件名` 或 `git add .`
提交修改	`git commit -m "说明"`
关联远程仓库	`git remote add origin git@github.com:用户名/仓库名.git`
首次推送	`git push -u origin main`
日常推送	`git push`
拉取远程更新	`git pull`
克隆远程仓库	`git clone git@github.com:用户名/仓库名.git`
新建并切换分支	`git checkout -b 分支名`
查看分支	`git branch`
合并分支	`git merge`
查看提交历史	`git log --oneline`
撤销工作区改动	`git restore 文件名`
取消暂存	`git restore --staged 文件名`
修改最近一次提交	`git commit --amend -m "新说明"`
回退到指定提交	`git reset --hard 提交号`

命令记不住没关系，把这张表贴在电脑旁边，用上三次就熟了。